

Summer Project

Name - Snigdha Stp

Roll Number - 2023102036

Objective 1: David Silvers Lectures

Lecture 1

RL is presented as a branch of machine learning where an agent learns to make decisions by interacting with an environment. Unlike other paradigms, RL uses only a "reward" signal for feedback, which is often delayed. The agent's actions influence the subsequent data it receives. The core principle is the **Reward Hypothesis**, which states that all goals can be framed as the maximization of cumulative future reward.

The fundamental interaction in RL involves an agent taking an action (A_t) in a given state, and the environment responding with a reward (R_{t+1}) and a new observation (O_{t+1}). The concept of **state** (S_t) is crucial; it's the information used to determine what happens next. An ideal state, known as a **Markov state**, captures all relevant information from the history, making the future independent of the past given the present.

An RL agent can be composed of one or more of three key components:

1. **Policy (π):** The agent's strategy, which maps states to actions. It can be deterministic ($a = \pi(s)$) or stochastic ($\pi(a|s) = P[A_t = a|S_t = s]$).
2. **Value Function ($v\pi(s)$):** A prediction of the expected future reward from a state, used to evaluate how good a state is. $v\pi(s) = E\pi[R_{t+1} + \gamma R_{t+2} + \dots | S_t = s]$
3. **Model:** The agent's internal representation of the environment, which predicts the next state and reward.

Finally, the lecture outlines key dichotomies within RL:

- **Learning vs. Planning:** Learning involves an unknown environment, while planning uses a known model.
- **Exploration vs. Exploitation:** The trade-off between exploring the environment to find new information and exploiting known information to maximize rewards.
- **Prediction vs. Control:** Prediction is about evaluating a given policy, whereas control is about finding the optimal policy.

Lecture 2

This lecture by David Silver, "Markov Decision Processes," provides the mathematical foundation for modeling environments in reinforcement learning. It builds up the concept of a Markov Decision Process (MDP) layer by layer.

1. **Markov Process (MP):** This is the simplest model, also known as a Markov Chain. It consists of a set of states (S) and a state transition matrix (P) that defines the probability of moving from one state to another. The key is the **Markov Property**, which states that the future is independent of the past, given the present. The current state contains all necessary information to predict the next state.
2. **Markov Reward Process (MRP):** An MRP extends a Markov Process by adding rewards. It is defined by states (S), a transition matrix (P), a reward function (R) that gives the expected immediate reward from a given state, and a discount factor (γ). The discount factor prioritizes immediate rewards over future ones. The central concept here is the **state-value function $v(s)$** , which represents the total expected discounted reward from a starting state ' s '. This value function is defined by the **Bellman Equation**: $v(s) = E[R_{t+1} + \gamma v(S_{t+1}) \mid S_t = s]$
3. **Markov Decision Process (MDP):** The full MDP introduces actions (A) to the MRP. An agent can now choose an action in a state, influencing the transitions and rewards. An MDP is defined by states (S), actions (A), a transition matrix (P), a reward function (R), and a discount factor (γ).

The agent's behavior is defined by a **policy $\pi(a|s)$** , which is a distribution over actions given a state. For any policy, there are two key value functions:

- **State-value function $v_{\pi}(s)$:** The expected return starting from state 's' and following policy π .
- **Action-value function $q_{\pi}(s, a)$:** The expected return from taking action 'a' in state 's' and then following policy π .

The ultimate goal in an MDP is to find the *optimal policy* (π^*) that maximizes the value function. This leads to the **Bellman Optimality Equation**, which states that the value of a state under an optimal policy must equal the expected return for the best action from that state. Solving these equations allows the agent to determine the best course of action in any given state.

Lecture 3

This lecture by David Silver, "Planning by Dynamic Programming," focuses on algorithms for solving Markov Decision Processes (MDPs) when a perfect model of the environment is known. Dynamic Programming (DP) is a powerful method for planning that breaks down complex problems into simpler, recursive subproblems. In the context of RL, this is achieved through the Bellman equations.

The lecture presents two main problems that DP can solve: prediction and control.

Prediction: Policy Evaluation

For the prediction problem, the goal is to evaluate a given policy, π . This means calculating the state-value function, $v_{\pi}(s)$, for every state. The solution is **Iterative Policy Evaluation**, which starts with an arbitrary value function and repeatedly applies the Bellman expectation backup to update the value of each state. For each state s , the update is:

$$v_{k+1}(s) = \sum \pi(a|s) (R(s,a) + \gamma \sum P(s'|s,a) v_k(s'))$$

This process is iterated until the value function converges.

Control: Finding the Optimal Policy

For the control problem, the goal is to find the optimal policy, π^* . The lecture introduces two main algorithms:

1. **Policy Iteration:** This algorithm alternates between two steps:
 - **Policy Evaluation:** Evaluate the current policy π to compute v_{π} .

- **Policy Improvement:** Improve the policy by acting greedily with respect to v_{π} . A new, better policy π' is created by choosing the action that maximizes the expected value from each state.

This cycle of evaluation and improvement is repeated until the policy no longer changes, at which point it has converged to the optimal policy, π^* .

2. **Value Iteration:** This algorithm is a more direct approach to finding the optimal value function, v^* . It combines the policy improvement and a single evaluation step into one update. It iteratively applies the Bellman optimality backup:

$$v_{k+1}(s) = \max_a (R(s,a) + \gamma \sum P(s'|s,a) v_k(s'))$$

Once the value function converges to v^* , the optimal policy can be extracted by choosing the action that maximizes the value in each state.

The lecture also discusses extensions like **Asynchronous DP**, which can reduce computation by updating states individually. Finally, it establishes the theoretical foundation for why these methods work, explaining that the Bellman backup operators are **contractions**. The **Contraction Mapping Theorem** guarantees that both policy evaluation and value iteration will converge to a unique optimal solution.

Lecture 4

To estimating its value function when the model (transition probabilities and rewards) is unknown. The agent must learn directly from experience. The lecture introduces two primary methods for this: Monte-Carlo Learning and Temporal-Difference Learning.

1. Monte-Carlo (MC) Learning

Monte-Carlo methods learn the value function by running complete episodes and averaging the returns.

- **Core Idea:** The value of a state, $V(s)$, is estimated as the empirical mean of the returns (G_t) that have been observed starting from that state. The return G_t is the total discounted reward from time-step t to the end of the episode.
- **Process:** After each episode, for every state S_t visited, the value function is updated towards the actual return G_t :

$$V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))$$

- **Properties:** MC has zero bias because it uses the true, complete return. However, it suffers from high variance because the return depends on all the random actions and transitions in the rest of the episode. A major limitation is that MC can only be used for episodic tasks, as it must wait until the end of an episode to compute the return.

2. Temporal-Difference (TD) Learning

Temporal-Difference learning improves upon MC by learning from incomplete episodes.

- **Core Idea:** TD learning updates the value of a state based on the value of the *next* state it observes. This process is called **bootstrapping**. Instead of waiting for the final return, TD(0) updates its estimate using the immediate reward plus the discounted value of the next state.
- **Process:** After each step (transition from S_t to S_{t+1} with reward R_{t+1}), the value function is updated:

$$V(S_t) \leftarrow V(S_t) + \alpha(R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$

The term $R_{t+1} + \gamma V(S_{t+1})$ is called the **TD target**, and the difference between the target and the current value is the **TD error**.

- **Properties:** TD has low variance because its update depends on only one random transition. However, it has some bias because the TD target uses an estimated value, $V(S_{t+1})$, which may be inaccurate. TD is generally more efficient than MC and can be used in continuous (non-terminating) environments.

3. TD(λ): Unifying MC and TD

TD(λ) provides a spectrum of methods that bridge the gap between one-step TD(0) and full-episode MC.

- **Forward View (Theoretical):** The **λ -return ($G_\lambda t$)** is a weighted average of all possible n -step returns, allowing for a mix of bootstrapping depths. When $\lambda=0$, we get the TD(0) return. When $\lambda=1$, we get the MC return.

- **Backward View (Practical):** This is the practical implementation of $TD(\lambda)$ using **eligibility traces ($E_t(s)$)**. An eligibility trace is a short-term memory that tracks which states are eligible for learning updates. It combines both the recency and frequency of visits. At each time step, the TD error is calculated, and the value function for every state is updated in proportion to its eligibility trace: $V(s) \leftarrow V(s) + \alpha \delta_t E_t(s)$. This allows credit for a reward to be propagated back through the recently visited states.

Lecture 5

The core of model-free control is **Generalized Policy Iteration (GPI)**, but adapted to work without a model. Since a model is needed to improve a policy from a state-value function $V(s)$, model-free methods must learn the **action-value function $Q(s,a)$** instead.

A key challenge is the **exploration-exploitation tradeoff**. If an agent only exploits its current knowledge by acting greedily, it may never discover better actions. To ensure all actions are tried, **ϵ -greedy exploration** is introduced, where the agent chooses a random action with a small probability ϵ and acts greedily otherwise.

The lecture divides model-free control into two main categories:

1. On-Policy Learning

In on-policy learning, the agent "learns on the job" by evaluating and improving the same policy it uses to act.

- **On-Policy Monte-Carlo Control:** Uses complete episodes to estimate $Q(s,a)$. The policy is then improved using ϵ -greedy exploration with respect to the current Q -values.
- **Sarsa (On-Policy TD Control):** This is the Temporal-Difference counterpart. It updates the Q -value at every time-step, which is more efficient. The update rule is based on the tuple $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$, from which the name Sarsa is derived:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t))$$

Because the update uses the *next action actually taken (A_{t+1})*, it is learning about the same ϵ -greedy policy it is following.

2. Off-Policy Learning

In off-policy learning, the agent learns about a different **target policy** (π) while following a **behavior policy** (μ). This allows the agent to learn about the optimal policy while still exploring.

- **Q-Learning (Off-Policy TD Control)**: This is a major breakthrough in reinforcement learning. Q-learning learns about the optimal (greedy) target policy, regardless of the behavior policy being followed. The update rule is:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma * \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t))$$

The key difference from Sarsa is the **max** operator, which finds the best possible next action for the update, rather than using the one the behavior policy actually chose. This allows Q-learning to find the optimal path, while an on-policy method like Sarsa might find a safer, less optimal path to account for its own exploration.

Lecture 6

This lecture by David Silver, "Value Function Approximation," addresses the critical challenge of applying reinforcement learning to problems with enormous or continuous state spaces, where using a simple lookup table for value functions is impossible.

The core solution is to **approximate** the value function using a parameterized function, $\hat{v}(s, w)$ or $\hat{q}(s, a, w)$, where w is a vector of parameters. This approach allows the agent to generalize from states it has visited to new, unseen states. The learning process then becomes about finding the best parameters w .

The lecture presents two main families of methods for training these parameters:

1. Incremental Methods

These methods update the parameter vector w after each experience, typically using **Stochastic Gradient Descent (SGD)**. The goal is to minimize the mean-squared error between the approximated value and a target value.

- The SGD update rule is: $\Delta w = \alpha * (\text{target} - \hat{v}(s, w)) * \nabla_w \hat{v}(s, w)$
- The **target** varies depending on the algorithm:

- **Monte-Carlo:** The target is the full return G_t . This is an unbiased but high-variance approach.
- **Temporal-Difference (TD):** The target is the TD target $R_{t+1} + \gamma \hat{v}(S_{t+1}, w)$. This is a biased, low-variance approach that allows for online learning.

A key point is that while MC is stable with function approximation, TD learning can become unstable and diverge, especially when used off-policy with non-linear approximators (a situation known as the "deadly triad").

2. Batch Methods

Instead of updating after every step, batch methods use a whole set of experiences to find the best-fitting parameters. These methods are generally more sample-efficient.

- The main idea is to formulate the problem as a **Least Squares** optimization, finding the parameters w that minimize the error over the entire batch of data.
- **Experience Replay** is a powerful technique that bridges the gap between incremental and batch methods. It stores experiences in a memory buffer and then samples mini-batches to train the function approximator. This technique is a cornerstone of **Deep Q-Networks (DQN)**, which use deep neural networks as function approximators and have achieved superhuman performance in domains like Atari games.
- For linear function approximation, batch methods like **Least Squares TD (LSTD)** and **Least Squares Policy Iteration (LSPI)** can solve for the optimal parameters directly and efficiently without iterative gradient updates.

Lecture 7

This lecture by David Silver, "Policy Gradient," introduces a new category of reinforcement learning algorithms that directly learn a parameterized policy, rather than learning a value function first.

From Value-Based to Policy-Based RL

Previous lectures focused on **value-based** methods (like Q-learning), where the agent learns an action-value function $Q(s, a)$ and the policy is implicitly derived (e.g., by acting greedily). In contrast, **policy-based** methods directly parameterize

the policy itself: $\pi_{\theta}(s, a) = P[a|s, \theta]$. The goal is to find the optimal parameters θ that maximize the expected reward.

Policy-based methods offer several advantages:

- They have better convergence properties.
- They are effective in continuous or high-dimensional action spaces.
- They can learn **stochastic policies**, which are optimal for certain problems where a deterministic policy would fail (e.g., in partially observable environments).

Policy Gradient Methods

The core idea is to adjust the policy parameters θ in the direction that increases the expected reward, a process known as **gradient ascent**. The **Policy Gradient Theorem** provides the theoretical foundation, showing that the gradient of the objective function $J(\theta)$ is:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(s, a) * Q \pi_{\theta}(s, a)]$$

This fundamental result leads to several algorithms:

1. **REINFORCE (Monte-Carlo Policy Gradient)**: This is the most basic policy gradient algorithm. It runs a full episode and then updates the policy parameters using the total return v_t as an unbiased sample of $Q \pi_{\theta}(s_t, a_t)$. While unbiased, this method suffers from very high variance because the return v_t depends on the entire rest of the episode.
2. **Actor-Critic Methods**: To reduce the high variance of REINFORCE, Actor-Critic methods are introduced. They consist of two components:
 - The **Actor** is the policy (π_{θ}), which decides which action to take. It updates its parameters θ using the policy gradient.
 - The **Critic** is a value function approximator that learns to estimate the action-value function $Q_w(s, a)$. It provides a lower-variance estimate of Q to the actor.

This setup allows the actor to get feedback at each step from the critic, rather than waiting until the end of an episode.

Improving Actor-Critic

To further improve performance, the actor-critic framework can be enhanced by using the **advantage function**, $A(s, a) = Q(s, a) - V(s)$. This function measures how much better an action is compared to the average action from that state. Using the advantage function reduces variance significantly. A practical way to estimate the advantage is by using the **TD error**, which acts as a low-variance (though biased) sample of the advantage. This leads to more stable and efficient learning.

Lecture 8

Model-Based vs. Model-Free Reinforcement Learning

The lecture begins by distinguishing between two main approaches in reinforcement learning:

- **Model-Free** : In this approach, an agent learns a value function and/or a policy directly from experience without creating a model of the environment.
- **Model-Based** : Here, the agent first learns a model of the environment from its experiences. It then uses this model to plan a course of action, which helps it determine a value function or a policy.

Understanding Model-Based Reinforcement Learning

A **model** is a representation of how the environment works. It predicts the next state and reward given the current state and an action. The goal is to learn a model that closely approximates the true environment.

Learning a model is treated as a supervised learning problem. For example, learning the reward function is a regression problem, while learning the state transitions is a density estimation problem. One simple type of model is a **Table Lookup Model**, where the agent keeps track of the outcomes of each state-action pair it has experienced. The transition probability and reward can be estimated by counting visits, as shown in the equations:

- Transition Probability: $P(s' \mid s, a) = (1 / N(s, a)) * \sum 1(St, At, St+1 = s, a, s')$

- Reward: $R(s, a) = (1 / N(s, a)) * \sum 1(St, At = s, a) * Rt$

Once a model is learned, an agent can use **planning** methods like value iteration or policy iteration on the model. A powerful approach is **sample-based planning**, where the agent uses the model to generate simulated experiences and then applies model-free learning algorithms like Q-learning to these simulated experiences.

The main advantage of model-based is that it can be more data-efficient. However, a key disadvantage is that the performance of the agent is limited by the accuracy of its learned model.

Integrated Architectures: The Dyna Framework

To get the best of both worlds, the lecture introduces integrated architectures that combine learning from real and simulated experiences. The **Dyna architecture** is a prominent example.

In Dyna, particularly the **Dyna-Q algorithm**, the agent learns from real experience as in model-free Q-learning. In addition, it uses this real experience to update its internal model. Then, it performs several steps of planning by randomly sampling from its model and applying Q-learning updates to the simulated experiences. This allows the agent to learn more from each real interaction with the environment.

Simulation-Based Search

Instead of learning a complete policy for the entire environment, an agent can use its model for **forward search** from the current state to find the best immediate action. **Simulation-based search** achieves this by simulating episodes starting from the current state.

A popular method is **Monte-Carlo Tree Search (MCTS)**. In MCTS, the agent builds a search tree from the current state. It simulates many episodes, and for each simulation, it updates the value of the actions in its search tree. The simulation policy itself is improved during this process. After the search is complete, the agent picks the action with the highest value in the tree. MCTS has been famously successful in complex games like Go.

Temporal-Difference Search

As an alternative to Monte-Carlo methods in search, **Temporal-Difference (TD) search** can be used. TD search applies methods like Sarsa to the sub-problem starting from the current state. While TD learning can introduce bias, it often reduces variance and can be more efficient than Monte-Carlo methods.

The **Dyna-2** architecture extends this idea by maintaining two sets of memory:

- **Long-term memory:** General knowledge learned from real experience using TD learning.
- **Short-term memory:** Specific knowledge about the current situation learned from simulated experience using TD search.

The overall value function is a combination of these two memories.

Objective 2: OpenAI Gym tool

1. Implementing an Agent that takes random actions:

For example, if we take a Cart Pole environment :

This environment helps to learn control systems through trial and error. The pole is attached to a cart that moves along a friction less track. We need to keep the pole balanced upright by applying forces to move the cart left or right.

- a. **Action Space:** The agent can take one of two actions at each step:
 - Push the cart to the **right** (action 1)
 - Push the cart to the **left** (action 0)
- b. **Observation Space:** The agent receives a state described by four values:
 - Pole angular velocity
 - Pole angle
 - Cart velocity
 - Cart position
- c. **Goal:** The agent must prevent the pole from falling over (i.e., keep it within a certain angle from vertical) and keep the cart within track boundaries for

as long as possible. Each time step where the pole remains upright, the agent receives a reward (typically +1), encouraging longer episodes

d. **Episode Termination:** The episode ends if:

- A maximum number of time steps is reached
- The cart moves out of bounds from centre.
- The pole falls past a certain angle

In the following code, the environment is initialised and interacts with it step by step, here the policy of the agent is random i.e it picks an action randomly from the available action space (either 0 or 1, corresponding to left or right).

```
import gym

env = gym.make("CartPole-v1", render_mode="human")

num_episodes = 5
total_rewards = []

for episode in range(num_episodes):
    state, _ = env.reset()
    print(f"\n=== Episode {episode + 1} ===")
    step = 0
    done = False
    episode_reward = 0

    while not done:
        action = env.action_space.sample()
        next_state, reward, terminated, truncated, info = env.step(action)
        done = terminated or truncated

    print(
        f"Step: {step} | "
        f"State: {state} | "
        f"Action: {action} | "
        f"Next State: {next_state} | "
```

```

        f"Reward: {reward} | "
        f"Done: {done}"
    )

    state = next_state
    episode_reward += reward
    step += 1

total_rewards.append(episode_reward)

if terminated:
    print("Episode ended: Pole fell or cart went out of bounds.")
elif truncated:
    print("Episode ended: Time limit (500 steps) reached.")

print(f"Episode {episode + 1} finished after {step} steps with total reward {

env.close()

avg_reward = sum(total_rewards) / num_episodes
print(f"\nAverage Reward over {num_episodes} episodes: {avg_reward:.2f}")

```

the environment is created by rendering and the results are as follows:

Step: 0	State: [-0.02500682 0.03811138 0.04174602 0.01489484]	Action: 0	Next State: [-0.0242446 -0.15758362 0.04204392 0.32045138]	Reward: 1.0	Done: False	Info: {}
Step: 1	State: [-0.0242446 -0.15758362 0.04204392 0.32045138]	Action: 1	Next State: [-0.02739627 0.03691513 0.04845295 0.04131833]	Reward: 1.0	Done: False	Info: {}
Step: 2	State: [-0.02739627 0.03691513 0.04845295 0.04131833]	Action: 0	Next State: [-0.02665797 -0.15886693 0.04927931 0.34888637]	Reward: 1.0	Done: False	Info: {}
Step: 3	State: [-0.02665797 -0.15886693 0.04927931 0.34888637]	Action: 0	Next State: [-0.0290353 -0.35405307 0.05025704 0.0566925]	Reward: 1.0	Done: False	Info: {}
Step: 4	State: [-0.0290353 -0.35405307 0.05025704 0.0566925]	Action: 0	Next State: [-0.03692838 -0.55051196 0.06939089 0.9665457]	Reward: 1.0	Done: False	Info: {}
Step: 5	State: [-0.03692838 -0.55051196 0.06939089 0.9665457]	Action: 1	Next State: [-0.04793862 -0.35638708 0.0887218 0.6964437]	Reward: 1.0	Done: False	Info: {}
Step: 6	State: [-0.04793862 -0.35638708 0.0887218 0.6964437]	Action: 0	Next State: [-0.05506636 -0.5526201 0.10265068 1.0156856]	Reward: 1.0	Done: False	Info: {}
Step: 7	State: [-0.05506636 -0.5526201 0.10265068 1.0156856]	Action: 0	Next State: [-0.06611876 -0.74894994 0.12296439 1.3387583]	Reward: 1.0	Done: False	Info: {}
Step: 8	State: [-0.06611876 -0.74894994 0.12296439 1.3387583]	Action: 1	Next State: [-0.08109776 -0.55557215 0.14973952 1.0869404]	Reward: 1.0	Done: False	Info: {}
Step: 9	State: [-0.08109776 -0.55557215 0.14973952 1.0869404]	Action: 0	Next State: [-0.09220921 -0.7523173 0.17147833 1.4226148]	Reward: 1.0	Done: False	Info: {}
Step: 10	State: [-0.09220921 -0.7523173 0.17147833 1.4226148]	Action: 1	Next State: [-0.10725555 -0.55968016 0.19993062 1.1880649]	Reward: 1.0	Done: False	Info: {}
Step: 11	State: [-0.10725555 -0.55968016 0.19993062 1.1880649]	Action: 0	Next State: [-0.11844915 -0.75675136 0.22369193 1.5361722]	Reward: 1.0	Done: True	Info: {}

Episode finished after 12 steps.

as here there is no policy this cannot maximise the cumulative reward.

Objective 3:

DDPG Algorithm

Deep Deterministic Policy Gradient (DDPG) is an off-policy, model-free reinforcement learning algorithm designed for environments with continuous action spaces.

It combines ideas from both Deep Q-Learning and Policy Gradient methods. DDPG is an actor-critic algorithm where the actor learns a deterministic policy (a mapping from states to actions), and the critic estimates the Q-value of state-action pairs. The key idea is to use two neural networks: the **actor network** for the policy and the **critic network** for the Q-value function. Additionally, DDPG maintains **target networks** (delayed copies of the actor and critic networks) to stabilise training by smoothing out rapid changes in value estimates.

To enhance exploration in continuous spaces, DDPG uses a noise process (like Gaussian noise) added to the actions. Experience replay is employed to break the correlation between consecutive samples by storing transitions in a buffer and sampling mini-batches randomly. During training, the critic is updated using the Bellman equation, and the actor is updated using the deterministic policy gradient, which maximizes the Q-value as predicted by the critic. Target networks are softly updated using a factor τ , which helps improve convergence stability.

Full DDPG implementation :

a. Objective:

Train an agent to swing a pendulum upright and keep it balanced (maximizing reward, minimizing torque and angle deviation) using **continuous control**.

b. the code

```
import gym
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import matplotlib.pyplot as plt
import random
from collections import deque

torch.manual_seed(0)
```

```

np.random.seed(0)
random.seed(0)

class Actor(nn.Module):
    def __init__(self, state_dim, action_dim, max_action):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, 256), nn.ReLU(),
            nn.Linear(256, 256), nn.ReLU(),
            nn.Linear(256, action_dim),
            nn.Tanh()
        )
        self.max_action = max_action

    def forward(self, state):
        return self.net(state) * self.max_action

class Critic(nn.Module):
    def __init__(self, state_dim, action_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim + action_dim, 256), nn.ReLU(),
            nn.Linear(256, 256), nn.ReLU(),
            nn.Linear(256, 1)
        )

    def forward(self, state, action):
        x = torch.cat([state, action], dim=1)
        return self.net(x)

class ReplayBuffer:
    def __init__(self, max_size=100000):
        self.buffer = deque(maxlen=max_size)

    def add(self, state, action, reward, next_state, done):
        self.buffer.append((state, action, reward, next_state, float(done)))

```



```

def sample(self, batch_size):
    batch = random.sample(self.buffer, batch_size)
    state, action, reward, next_state, done = map(np.array, zip(*batch))
    return map(lambda x: torch.FloatTensor(x), (state, action, reward, next_state, done))

```

```

class DDPGAgent:

```

```

    def __init__(self, env, gamma=0.99, tau=0.005, actor_lr=1e-3, critic_lr=1e-3):

```

```

        self.env = env

```

```

        self.state_dim = env.observation_space.shape[0]

```

```

        self.action_dim = env.action_space.shape[0]

```

```

        self.max_action = float(env.action_space.high[0])

```

```

        self.actor = Actor(self.state_dim, self.action_dim, self.max_action)

```

```

        self.actor_target = Actor(self.state_dim, self.action_dim, self.max_action)
    on)

```

```

        self.actor_target.load_state_dict(self.actor.state_dict())

```

```

        self.critic = Critic(self.state_dim, self.action_dim)

```

```

        self.critic_target = Critic(self.state_dim, self.action_dim)

```

```

        self.critic_target.load_state_dict(self.critic.state_dict())

```

```

        self.actor_optimizer = optim.Adam(self.actor.parameters(), lr=actor_lr)
    r)

```

```

        self.critic_optimizer = optim.Adam(self.critic.parameters(), lr=critic_lr)

```

```

        self.replay_buffer = ReplayBuffer()

```

```

        self.gamma = gamma

```

```

        self.tau = tau

```

```

    def select_action(self, state, noise_scale=0.1):

```

```

        state_tensor = torch.FloatTensor(state.reshape(1, -1))

```

```

        action = self.actor(state_tensor).detach().cpu().numpy()[0]

```

```

        return np.clip(action + noise_scale * np.random.randn(*action.shape), -self.max_action, self.max_action)

    def train(self, batch_size=64):
        if len(self.replay_buffer.buffer) < batch_size:
            return

        state, action, reward, next_state, done = self.replay_buffer.sample(batch_size)

        # Critic loss
        with torch.no_grad():
            next_action = self.actor_target(next_state)
            target_Q = reward.unsqueeze(1) + self.gamma * (1 - done.unsqueeze(1)) * self.critic_target(next_state, next_action)

        current_Q = self.critic(state, action)
        critic_loss = nn.MSELoss()(current_Q, target_Q)

        self.critic_optimizer.zero_grad()
        critic_loss.backward()
        self.critic_optimizer.step()

        # Actor loss
        actor_loss = -self.critic(state, self.actor(state)).mean()

        self.actor_optimizer.zero_grad()
        actor_loss.backward()
        self.actor_optimizer.step()

        # Soft update target networks
        for param, target_param in zip(self.actor.parameters(), self.actor_target.parameters()):
            target_param.data.copy_(self.tau * param.data + (1 - self.tau) * target_param.data)

```

```

        for param, target_param in zip(self.critic.parameters(), self.critic_target.parameters()):
            target_param.data.copy_(self.tau * param.data + (1 - self.tau) * target_param.data)

    def run(self, episodes=100, max_steps=200):
        rewards = []

        for ep in range(episodes):
            state, _ = self.env.reset()
            ep_reward = 0
            for t in range(max_steps):
                action = self.select_action(state)
                next_state, reward, done, _, _ = self.env.step(action)
                self.replay_buffer.add(state, action, reward, next_state, done)
                state = next_state
                ep_reward += reward

            self.train()

            if done:
                break
            print(f"Episode {ep+1}: Return = {ep_reward:.2f}")
            rewards.append(ep_reward)

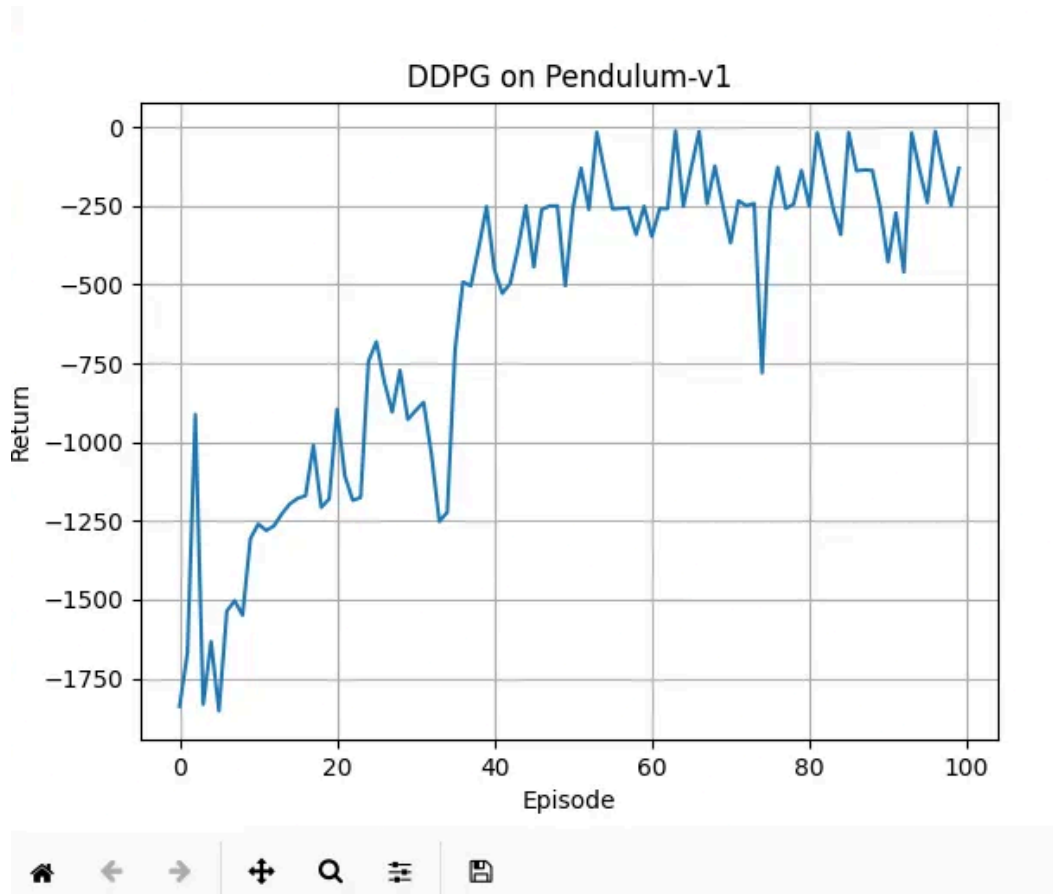
        return rewards

env = gym.make("Pendulum-v1")
agent = DDPGAgent(env)
rewards = agent.run()

plt.plot(rewards)
plt.title("DDPG on Pendulum-v1")
plt.xlabel("Episode")
plt.ylabel("Return")

```

```
plt.grid()
plt.show()
```



Over 100 episodes, the DDPG agent showed consistent improvement on Pendulum-v1. Starting with poor returns (around -1800), it steadily improved to reach returns closer to -100. The trend indicates successful learning and convergence, though occasional fluctuations reflect the inherent instability in off-policy actor-critic methods. The final performance demonstrates that DDPG effectively learned a near-optimal policy to balance the pendulum.

PPO Algorithm :

Proximal Policy Optimization (PPO) is a reinforcement learning algorithm that improves an agent's behavior through **policy gradient methods**. It uses an **actor-**

critic framework, where the actor (policy network) decides actions and the critic (value network) evaluates them. PPO prevents drastic changes to the policy by **clipping the policy update**, making training more stable. It's widely used due to its simplicity, efficiency, and strong performance in both discrete and continuous environments.

```
import gym
import numpy as np
import matplotlib.pyplot as plt

class Policy:
    def __init__(self, state_dim, action_dim, hidden_size=32):
        self.state_dim = state_dim
        self.action_dim = action_dim

        # Initialize weights for a simple 2-layer neural net
        self.w1 = np.random.randn(state_dim, hidden_size) * 0.1
        self.b1 = np.zeros(hidden_size)
        self.w2 = np.random.randn(hidden_size, action_dim) * 0.1
        self.b2 = np.zeros(action_dim)

        # For tracking gradients (manual backprop)
        self.lr = 1e-2

    def forward(self, state):
        z1 = np.dot(state, self.w1) + self.b1
        h1 = np.tanh(z1)
        z2 = np.dot(h1, self.w2) + self.b2
        exp = np.exp(z2 - np.max(z2)) # for numerical stability
        probs = exp / np.sum(exp)
        return probs, h1

    def sample_action(self, state):
        probs, _ = self.forward(state)
        action = np.random.choice(len(probs), p=probs)
```

```

    return action, probs

def update(self, states, actions, advantages, old_probs):
    for state, action, adv, old_p in zip(states, actions, advantages, old_probs):
        probs, h1 = self.forward(state)
        ratio = probs[action] / (old_p[action] + 1e-10)
        clipped = np.clip(ratio, 1 - 0.2, 1 + 0.2)
        grad_coeff = -min(ratio * adv, clipped * adv)

        # Manual gradients
        dlogits = probs
        dlogits[action] -= 1
        dlogits *= grad_coeff

        grad_w2 = np.outer(h1, dlogits)
        grad_b2 = dlogits

        dh1 = np.dot(self.w2, dlogits)
        dz1 = dh1 * (1 - h1 ** 2)

        grad_w1 = np.outer(state, dz1)
        grad_b1 = dz1

        # Gradient descent
        self.w1 -= self.lr * grad_w1
        self.b1 -= self.lr * grad_b1
        self.w2 -= self.lr * grad_w2
        self.b2 -= self.lr * grad_b2

def discount_rewards(rewards, gamma=0.99):
    discounted = np.zeros_like(rewards, dtype=np.float32)
    G = 0
    for i in reversed(range(len(rewards))):
        G = rewards[i] + gamma * G
        discounted[i] = G
    return discounted

```

```

def run_ppo():
    env = gym.make("CartPole-v1")
    policy = Policy(state_dim=4, action_dim=2)
    all_rewards = []

    for episode in range(200):
        state, _ = env.reset()
        done = False

        states = []
        actions = []
        rewards = []
        old_probs = []

        while not done:
            action, probs = policy.sample_action(state)
            next_state, reward, done, _, _ = env.step(action)

            states.append(state)
            actions.append(action)
            rewards.append(reward)
            old_probs.append(probs)

            state = next_state

        total_reward = sum(rewards)
        all_rewards.append(total_reward)

        returns = discount_rewards(rewards)
        advantages = returns - np.mean(returns)
        advantages /= np.std(advantages) + 1e-8

        policy.update(states, actions, advantages, old_probs)

    print(f"Episode {episode+1}, Return: {total_reward}")

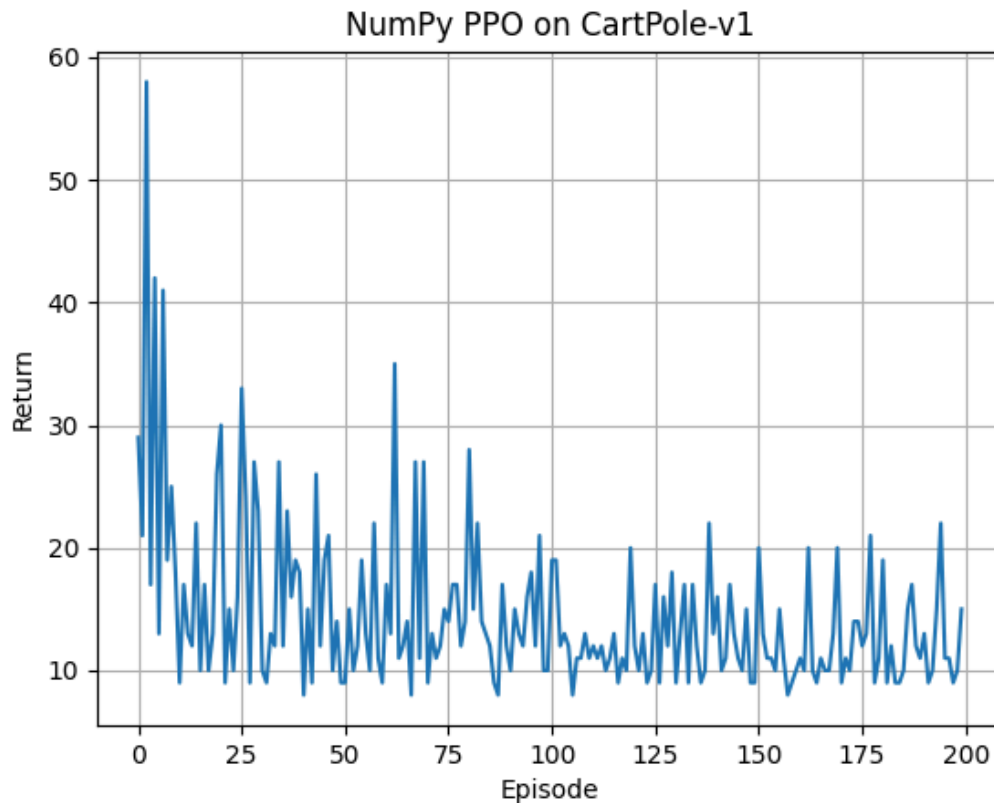
```

```
    return all_rewards

rewards = run_ppo()

plt.plot(rewards)
plt.title("NumPy PPO on CartPole-v1")
plt.xlabel("Episode")
plt.ylabel("Return")
plt.grid()
plt.show()
```

```
Episode 1, Return: 18.0
Episode 2, Return: 19.0
Episode 3, Return: 14.0
Episode 4, Return: 13.0
Episode 5, Return: 15.0
Episode 6, Return: 12.0
Episode 7, Return: 17.0
Episode 8, Return: 10.0
Episode 9, Return: 14.0
Episode 10, Return: 20.0
Episode 11, Return: 17.0
Episode 12, Return: 27.0
Episode 13, Return: 9.0
Episode 14, Return: 19.0
Episode 15, Return: 20.0
Episode 16, Return: 27.0
Episode 17, Return: 52.0
Episode 18, Return: 17.0
Episode 19, Return: 29.0
Episode 20, Return: 12.0
Episode 21, Return: 14.0
Episode 22, Return: 23.0
Episode 23, Return: 21.0
Episode 24, Return: 18.0
Episode 25, Return: 19.0
Episode 26, Return: 16.0
Episode 27, Return: 15.0
Episode 28, Return: 13.0
Episode 29, Return: 13.0
```

This plot tells us the current **NumPy PPO agent is not effectively learning** the CartPole task.

- It has **high variance, poor convergence, and low final performance.**
- The agent may be **overfitting early experiences** or **updating too aggressively.**
- Fixes include **better policy structure, critic network, and tuned hyper-parameters.**

The below Implementation :

```
import gym
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import matplotlib.pyplot as plt
```

```

# Actor-Critic Network
class Actor(nn.Module):
    def __init__(self, state_dim, action_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, 64), nn.Tanh(),
            nn.Linear(64, 64), nn.Tanh(),
            nn.Linear(64, action_dim), nn.Softmax(dim=-1)
        )

    def forward(self, x):
        return self.net(x)

class Critic(nn.Module):
    def __init__(self, state_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, 64), nn.Tanh(),
            nn.Linear(64, 64), nn.Tanh(),
            nn.Linear(64, 1)
        )

    def forward(self, x):
        return self.net(x)

# PPO Agent
class PPOAgent:
    def __init__(self, env_name="CartPole-v1", gamma=0.99, eps_clip=0.2, lr=2.
5e-4):
        self.env = gym.make(env_name)
        self.state_dim = self.env.observation_space.shape[0]
        self.action_dim = self.env.action_space.n

        self.actor = Actor(self.state_dim, self.action_dim)
        self.critic = Critic(self.state_dim)

```

```

self.optimizerA = optim.Adam(self.actor.parameters(), lr=lr)
self.optimizerC = optim.Adam(self.critic.parameters(), lr=lr)
self.gamma = gamma
self.eps_clip = eps_clip

def select_action(self, state):
    state = torch.FloatTensor(state).unsqueeze(0)
    probs = self.actor(state)
    dist = torch.distributions.Categorical(probs)
    action = dist.sample()
    return action.item(), dist.log_prob(action), dist.entropy()

def compute_returns_advantages(self, rewards, values, dones):
    returns, advantages = [], []
    G = 0
    A = 0
    values = values + [0] # bootstrap
    for t in reversed(range(len(rewards))):
        delta = rewards[t] + self.gamma * values[t + 1] * (1 - dones[t]) - values
[t]
        A = delta + self.gamma * 0.95 * (1 - dones[t]) * A # GAE
        G = rewards[t] + self.gamma * (1 - dones[t]) * G
        returns.insert(0, G)
        advantages.insert(0, A)
    return torch.tensor(returns), torch.tensor(advantages)

def train(self, episodes=200, epochs=4, batch_size=500):
    all_returns = []
    for episode in range(episodes):
        states, actions, log_probs, rewards, dones, values = [], [], [], [], [], []

        state, _ = self.env.reset()
        ep_reward = 0

        for _ in range(batch_size):
            action, log_prob, _ = self.select_action(state)

```

```

value = self.critic(torch.FloatTensor(state).unsqueeze(0)).item()

next_state, reward, done, _, _ = self.env.step(action)

states.append(torch.FloatTensor(state))
actions.append(action)
log_probs.append(log_prob)
rewards.append(reward)
dones.append(done)
values.append(value)

state = next_state
ep_reward += reward

if done:
    state, _ = self.env.reset()

returns, advantages = self.compute_returns_advantages(rewards, values, dones)
returns = (returns - returns.mean()) / (returns.std() + 1e-8)
advantages = (advantages - advantages.mean()) / (advantages.std() + 1e-8)

# Update policy and value networks
for _ in range(epochs):
    for i in range(0, len(states), 64):
        idx = slice(i, i+64)
        s_batch = torch.stack(states[idx])
        a_batch = torch.tensor(actions[idx])
        old_logp = torch.stack(log_probs[idx]).detach()
        adv_batch = advantages[idx].detach()
        ret_batch = returns[idx].detach()

        # Actor loss
        probs = self.actor(s_batch)
        dist = torch.distributions.Categorical(probs)

```

```

        logp = dist.log_prob(a_batch)
        ratio = torch.exp(logp - old_logp)
        surr1 = ratio * adv_batch
        surr2 = torch.clamp(ratio, 1 - self.eps_clip, 1 + self.eps_clip) * adv_
batch
        actor_loss = -torch.min(surr1, surr2).mean()

        # Critic loss
        value_pred = self.critic(s_batch).squeeze()
        critic_loss = nn.MSELoss()(value_pred, ret_batch)

        # Backprop
        self.optimizerA.zero_grad()
        actor_loss.backward()
        self.optimizerA.step()

        self.optimizerC.zero_grad()
        critic_loss.backward()
        self.optimizerC.step()

        all_returns.append(ep_reward)
        print(f"Episode {episode+1}, Reward: {ep_reward}")

    return all_returns

# Run PPO
ppo = PPOAgent()
returns = ppo.train()

# Plot
plt.plot(returns)
plt.title("PPO on CartPole-v1")
plt.xlabel("Episode")
plt.ylabel("Return")

```

```
plt.grid()  
plt.show()
```

the features of the above implementation are:

- **Generalized Advantage Estimation (GAE):** Stabilizes learning.
- **Clipping (ϵ):** Prevents large updates, improving stability.
- **Separate Actor/Critic networks:** Better specialization and optimization.
- **Normalized returns and advantages:** Helps convergence.
- **Minibatch training over multiple epochs.**